

Forward Deployed Engineer

Build an AI Agent for Client Data Migration & Integration

1. Overview

This task is an independent exercise — completed on your own, with no support available, and evaluated in a follow-up conversation with the panel.

You are not being tested on your ability to write code by hand. You are being tested on how you scope an agent's autonomy in a customer-facing scenario: what it should decide on its own, what it should escalate, and how you build a UI that lets a human supervise that without micromanaging it.

2. The challenge

A (fictitious) client is migrating from their old HR/CRM tool into a new platform. You'll be given a small set of the client's raw data exports (CSV/Excel, inconsistent column names, mixed date formats, duplicate records, missing fields) and a target schema (a simple JSON/YAML spec you define or we provide). Build an agent that reads the source files, works out the mapping to the target schema, cleans and validates the data, and pushes the result to a mock target API on its own — stopping to ask a human only when it hits something it's genuinely not confident about. Wrap it in a small web UI that a non-technical implementation consultant can use to watch the agent work, review what it's uncertain about, approve or correct those cases, and see a clear record of everything it did.

3. Acceptance criteria

- 1 Multi-file ingestion - the agent ingests multiple source files representing the same entity (e.g. "employees") with different column names/formats and reconciles them into one dataset without being told how, field by field.
- 2 Autonomous mapping and cleanup - the agent proposes and applies the source-to-target field mapping, fixes what it safely can (date formats, obvious duplicates, whitespace/casing), and does this without a human confirming every field.
- 3 Defensible escalation boundary - the agent only surfaces a case to the human when it's actually ambiguous (a column that could map to two target fields, a record that fails validation twice, a value it can't confidently clean) - not everything, and not nothing. Be ready to explain why you drew the line where you did.
- 4 Human-in-the-loop UI - a live view of what the agent is doing, a queue of its escalations with enough context to resolve them in one glance, and the ability to approve, correct, or reject.
- 5 Mock system integration - a "push to target" step that calls a stub API, reports success/failure per record, and supports retry or rollback on failure, with an audit trail of what changed and why.
- 6 Delta solutioning on top of what AI does.

4. Constraints & tooling

Use open-source AI models and AI coding tools to build this. You may also use AI coding tools (Claude, ChatGPT, Cursor, Copilot, etc.) to build, debug, and refine it.

5. Deliverables

- **Working prototype** - a hosted link, or local run instructions with a short demo recording (the recording should show at least one escalation getting resolved through the UI).
- **Source code repository** - a Git repo with the full source and a README covering setup and tech stack.
- **Short write-up (1 page max)** - your approach, and specifically how you decided what the agent should handle alone versus escalate, plus what you'd build next.

6. Time expectation

Keep this small. Most candidates should be able to complete it in roughly 4-6 focused hours - a smaller, well-thought-through agent beats a sprawling, half-working one.